

OS-Lab6

思考题

Thinking6.1

- 导致读者无法读到文件尾：管道的读端在所有写端关闭时才会返回 0（EOF）。如果读者进程保留了自己的写端未关闭，即使真正的写者已经退出，读端依然认为系统中存在潜在的写者，从而导致 read 操作阻塞，产生死锁。
- 资源泄露，内存页无法及时回收。

Thinking6.2

需要交换 `switch(fork())` 中父子进程的分支逻辑：

```

int fildes[2];
char buf[100];
int status;

int main(){
    status = pipe(fildes);
    if (status == -1 ) {
        printf("error\n");
    }

    switch (fork()) {
        case -1:
            break;

        case 0:
            close(fildes[1]);
            read(fildes[0], buf, 100);
            printf("child-process read:%s", buf);
            close(fildes[0]);
            exit(EXIT_SUCCESS);

        default:
            close(fildes[0]);
            write(fildes[1], "Hello world\n", 12);
            close(fildes[1]);
            exit(EXIT_SUCCESS);
    }
}

```

Thinking6.3

- 进程在读者和写者间频繁切换，切换效率降低
- 频繁阻塞切换，吞吐量下降

Thinking6.4

并非所有时刻成立。在执行 `close`、`dup` 或 `fork` 等涉及修改映射关系的操作时，由于 `unmap` 或 `map` 多个页面不是一个原子化的硬件操作，中间可能被时钟中断打断。

Thinking6.5

`dup(oldfd, newfd)` 的过程涉及先解除 `newfd` 的映射，再将 `oldfd` 的页面映射到 `newfd`。在映射过程中，需要分别对 FD 页面和 Pipe 数据页面进行操作。如果操作顺序不当，或者读者在中间状态读取引用计数，就会破坏恒等式平衡，导致逻辑判断失效。

Thinking6.6

原因：在 MOS 的单核模型中，系统调用通过 `syscall` 指令陷入内核，内核在处理异常时通常是关闭中断的。一旦进入内核执行系统调用代码，除非主动放弃 CPU，否则不会被其他用户进程打断。

反例：`sys_ipc_recv`，`sys_yield`，内部显式调用了 `schedule()`

Thinking6.7

能解决竞争。先 `unmap` FD 页面，再 `unmap` Pipe 页面。在任何时刻，`pageref(pipe)` 总是大于等于 `pageref(fd)`。

有类似问题。在 `dup` 时，应该先映射 Pipe 页面，再映射 FD 页面，确保 Pipe 的引用计数永远领先或等于 FD。

Thinking6.8

- `elf_load_seg` 先拷贝 `filesz` 长度的数据。
- 对于 `memsz > filesz` 的部分，`load_icode_mapper` 在分配物理页后，会将超出 `filesz` 的部分手动清零，从而保证未初始化的全局变量初始值为 0。

Thinking6.9

在 `init.b` 进程中。内核启动的第一个用户进程 `icode.b` 产生 `init.b`，`init.b` 首先调用 `opencons()` 打开控制台。紧接着通过 `dup(0, 1)` 将 0 号描述符复制给 1 号，1 就成了 `stdout`。后续所有的子进程都继承了这两个 FD。

Thinking6.10

MOS:外部命令。shell解析命令后通过 `spawn` 加载 *.b 文件执行。

Linux的 `cd`：因为进程的工作目录是进程属性。如果 `cd` 是外部命令，会在子进程中改变目录然后退出，而父进程的工作目录不会受到任何影响。只有作为内置命令在 shell 进程自身上下文中执行，才能改变 shell 的目录。

Thinking6.11

- spawn 次数：2次。
 - Shell为执行管道左侧命令 spawn 了 ls.b
 - Shell为执行管道右侧命令 spawn 了 cat.b
- 进程销毁次数：2次。
 - ls.b
 - cat.b

难点分析

本次作业涉及到对前面Lab，尤其是Lab5的综合运用，需要对文件管理系统有足够的了解。

核心难点有三：

- 理解利用 PTE_LIBRARY 权限位实现跨进程物理页共享的底层管道机制
- 掌握基于页引用计数的判等逻辑来识别管道关闭状态，并需通过严谨调整 unmap/map 操作顺序及引入 env_runs 变量，来解决由于非原子化操作和时钟中断引发的进程竞争问题
- 在 Shell 的实现中，不仅要处理复杂的文件描述符（FD）重定向逻辑，还需深入理解 spawn 函数如何通过手动构造用户栈空间（填充 argc/argv）**来完成新进程的初始化与加载。

实验体会

Lab6不用上机，所以不会让人感到太大心理压力。不过我还是把Lab6的exam和extra题型看了一下，毕竟Lab6是作为用户和操作系统交互最可视化的一次，完成题目的感觉还是比较让人有成就感的。

纵观OS课程7个Lab，虽然只做了一些补全代码工作，但也确实夯实了阅读代码和微调代码的功底，只是少量的修改毕竟让人感觉参与不足。害怕但期待挑战性任务中.....